

Triangulating Plane Algebraic Curves Intrinsically in $\mathbb{CP}^2$

Vinícius Moreira Mello

Departamento de Matemática, Universidade Federal da Bahia, Salvador, Brazil
vinicius.mello@ufba.br

Draft v3 – October 7, 2026

Abstract

A smooth plane algebraic curve is a compact Riemann surface and a real surface of codimension 2 in $\mathbb{CP}^2$. Most methods that mesh it work in an affine chart or over a branched covering of $\mathbb{CP}^1$, and must therefore treat points at infinity, charts or branch cuts separately. We triangulate the curve directly inside $\mathbb{CP}^2$, with no distinguished chart. A 15-vertex triangulation of $\mathbb{CP}^2$, due to Gaifullin, is refined adaptively by Maubach bisection along Fubini–Study geodesics, and the curve is followed from cell to cell by continuation. In each cell, the curve meets the triangular 2-faces in isolated points, which are joined into polygons. Realizing every face as a geodesic cone, rather than as a flat triangle in an affine chart, makes the faces shared by neighboring cells coincide exactly, so the pieces glue into a closed surface, whose topology is checked a posteriori. On our test curves at depth 14, fewer than 0.14% of the cells that meet the curve fail, against 1.8–11.6% with chart-flat faces; the area agrees with Wirtinger’s theorem to within 0.15%; and, unlike a box in an affine chart at equal resolution, the mesh covers the whole curve.

Contents

1	Introduction	2
2	Related work	3
3	Preliminaries	4
3.1	The Fubini–Study geometry of $\mathbb{CP}^2$	4
3.2	Maubach bisection	5
4	An adaptive mesh of $\mathbb{CP}^2$	5
4.1	The seed	5
4.2	Pointerless codes on balanced seeds	6
5	Extraction and continuation	9
5.1	Realizing the faces	9
5.2	Per-cell extraction	10
5.3	Continuation	11
5.4	Refinement near tangencies	12
6	Topological verification and closing	12

7	Experiments	12
7.1	Chart-flat faces versus geodesic cones	13
7.2	Bad-cell rate	13
7.3	Topology	14
7.4	Accuracy	15
7.5	Higher degree	15
7.6	Cost	16
7.7	Comparison with an affine chart	17
7.8	Comparison with $\mathbb{CP}^1 \times \mathbb{CP}^1$	18
8	Visualization	19
9	Limitations and conclusion	20

1 Introduction

Let $F(X, Y, Z)$ be a homogeneous polynomial of degree n such that $C = \{F = 0\} \subset \mathbb{CP}^2$ is smooth. Then C is a compact Riemann surface of genus $g = (n-1)(n-2)/2$, and, as a real object, a closed orientable surface of codimension 2 in the real 4-manifold $\mathbb{CP}^2$. This paper is about computing a triangulation of C , a simplicial surface close to C with the right topology, for visualization and as a combinatorial model.

Existing methods each treat some part of C specially.

- *Affine charts.* Meshing $\{F(x, y, 1) = 0\}$ in a grid of $\mathbb{C}^2 = \mathbb{R}^4$ [44] loses the n points at infinity, and in a bounded box it loses a neighborhood of them. Covering $\mathbb{CP}^2$ needs three charts, whose meshes must then be stitched along the overlaps.
- *Branched coverings.* Viewing C as an n -sheeted covering of $\mathbb{CP}^1$ [14, 38] requires the branch points, cuts between them, and a separate treatment of the fiber over ∞ .
- *Other compactifications.* Embedding $\mathbb{C}^2$ in $\mathbb{CP}^1 \times \mathbb{CP}^1$ instead of $\mathbb{CP}^2$ can create singularities that the curve does not have: the closure of $w^2 = z^3 - z$ has a cusp at (∞, ∞) in $\mathbb{CP}^1 \times \mathbb{CP}^1$, while its closure $X^2Z = Y^3 - YZ^2$ in $\mathbb{CP}^2$ is smooth. On the same code, extraction failures concentrate at that cusp (Section 7.8).

In $\mathbb{CP}^2$ the smooth curve is a smooth compact submanifold, with no distinguished points. A method that works intrinsically in $\mathbb{CP}^2$ needs none of these special cases.

Approach. We triangulate $\mathbb{CP}^2$ itself and refine it adaptively around C .

- The seed is Gaifullin’s 15-vertex triangulation of $\mathbb{CP}^2$ [21]. It is balanced, and therefore a valid initial mesh for Maubach bisection (Section 4).
- Refinement is Maubach bisection [31, 43], with new vertices at Fubini–Study geodesic midpoints.
- The mesh is pointerless: a cell is a seed index plus an LPT code [4], and neighbors are computed, not stored (Section 4.2).
- The curve is followed by continuation and extracted cell by cell from Newton crossings on triangular 2-faces, each realized as a geodesic cone so that faces glue exactly along their edges (Section 5).
- The topology of the result is checked, and holes are closed (Section 6).

Contributions.

1. An intrinsic pipeline that triangulates smooth plane curves inside $\mathbb{CP}^2$, with no charts, cuts or special treatment of infinity, built from Maubach bisection on a balanced seed of $\mathbb{CP}^2$, codimension-2 extraction and combinatorial continuation.
2. An extension of LPT codes from the Kuhn cube to arbitrary balanced seeds, including seeds that triangulate closed manifolds. The neighbor procedure is fully combinatorial and table-driven, and it is verified exactly. We also compare its memory use with an explicit incidence structure.
3. A chart-free realization of the triangular faces as Fubini–Study geodesic cones. Faces then glue exactly along geodesic edges, so the boundary of every tetrahedral facet is a closed surface and meets C an even number of times. Compared with faces that are flat in affine charts, this cuts extraction failures by a factor of 26 to 175 on our test curves.
4. An a-posteriori topological check and a closing step that turn an uncertified extraction into a closed surface of the right Euler characteristic, together with a global accuracy check by Wirtinger’s theorem.
5. An experimental comparison with the affine-chart approach on the same code base, at equal Fubini–Study resolution: a box in a chart misses neighbourhoods of the points at infinity, its missing area decreasing only like $1/R^2$ with its half-width R , and at $R = 16$ it already uses more cells than the whole of $\mathbb{CP}^2$.
6. A comparison with $\mathbb{CP}^1 \times \mathbb{CP}^1$ on the same code base, whose balanced seed is the product of two octahedra. Where this compactification gives the curve a cusp, extraction failures concentrate at it, and some polygons cut across it with the wrong orientation. A test with the Kähler form, $|\omega|/\text{area}$ per polygon, detects them, after which the closed surface has the right topology.

2 Related work

Computing with compact Riemann surfaces. Algebraic curves are usually handled numerically as branched coverings of $\mathbb{CP}^1$, to compute monodromy, periods and related invariants [14, 7]. Visualizations project the curve into a chart: Hanson’s Fermat surfaces [24] use an explicit parametrization; Nieser, Poelke and Polthier [38] mesh the multivalued graph over $\mathbb{C}$ from the branch points; Kranich [28] colors domains on the GPU; and Dutter’s chart-free map [17] displays the whole of $\mathbb{CP}^2$ in a bounded region of $\mathbb{R}^3$. Bertini_real [10] computes, by homotopy continuation, a cell decomposition of the real points of a complex algebraic set of dimension one or two. Writing $F = 0$ as the two real equations $\text{Re } F = \text{Im } F = 0$ in an affine chart makes the curve a real surface in $\mathbb{R}^4$ to which it applies, but in one chart at a time.

Implicit manifolds of codimension two. Our extraction relates to piecewise-linear approximation of implicit manifolds over Freudenthal–Kuhn triangulations [3, 2], to its certified, uniform-grid variant by cell-to-cell tracing [8, 9], and most directly to the extraction of zero sets of functions $\mathbb{C}^2 \rightarrow \mathbb{C}$ on uniform grids of $\mathbb{R}^4$ [44], which is the baseline of Section 7.7. Isosurface extraction over bisection hierarchies is common in codimension one [22, 39]; adaptive simplicial grids for implicit complexes in $\mathbb{R}^3$, including curve networks, are recent [26]. Castelo et al. [13] triangulate implicit manifolds adaptively in any dimension with the J1a triangulation, and Bhaniramka, Wenger and Crawfis [6] extract isosurfaces in any dimension with guarantees on the output, in codimension one.

Certification. Interval and Bernstein subdivision certify the topology of implicit curves and surfaces [40, 37]; on simplices, barycentric Bernstein bounds play the same role [41]. Our method is not certified: it reports the cells where extraction fails and checks the topology of the result a posteriori.

Bisection and initial meshes. Newest-vertex bisection goes back to Mitchell [35, 36] in 2D, and was extended to arbitrary dimension by Maubach [31] and Traxler [43]. Its weak point is the initial mesh, which must carry compatible vertex orders. Remedies include pre-refinement [27, 42], renumbering [1], marked-bisection stages [5], and the coloring-based initialization of Diening, Gehring and Storn [15]. All of them concern domains in $\mathbb{R}^d$. We use a seed of a closed manifold, which is admissible because it is balanced (Proposition 3.1).

Pointerless hierarchical meshes. Several representations encode a simplex of a regular bisection hierarchy by a label instead of pointers: Hebert [25]; Evans, Kirkpatrick and Townsend [20]; Lee, De Floriani and Samet [30]; Maubach’s own neighbor location [32]; and Atalay and Mount’s LPT codes [4] in arbitrary dimension, which we previously used for adaptive cellular structures in 3D printing [18]. Weiss and De Floriani use the star of the bisection edge (a *diamond*) as primitive [45]; Dupuy’s concurrent binary trees implement longest-edge bisection on the GPU [16]; Boissonnat, Kachanovich and Wintraecken encode Freudenthal–Kuhn simplices compactly, without adaptivity [8]. Architecturally closest to our construction are the forests of p4est [12] and t8code [11]: an arbitrary coarse mesh refined tree by tree, with a per-tree linear code and an inter-tree connectivity table. Those forests are non-conforming (hanging nodes, red refinement, $d \leq 3$); ours is a conforming, graded bisection forest in arbitrary dimension.

3 Preliminaries

3.1 The Fubini–Study geometry of $\mathbb{CP}^2$

We write points of $\mathbb{CP}^2$ as unit vectors $p \in \mathbb{C}^3$, defined up to a phase, with the Hermitian product $\langle p, q \rangle = \sum_k p_k \bar{q}_k$. The Fubini–Study distance is

$$d(p, q) = \arccos |\langle p, q \rangle| \in [0, \frac{\pi}{2}].$$

Linear combinations of representatives are not well defined on $\mathbb{CP}^2$: q and $-q$ are the same point, yet $(p + q)/2$ and $(p - q)/2$ are different points, and the second vanishes when $q = p$. So, to average two points, we first align their phases. If $\langle p, q \rangle \neq 0$, that is, if $d(p, q) < \frac{\pi}{2}$, the representative of q aligned to p is

$$\tilde{q} = e^{i\theta} q, \quad e^{i\theta} = \frac{\overline{\langle q, p \rangle}}{|\langle q, p \rangle|},$$

the unique representative of the point q with $\langle \tilde{q}, p \rangle$ real and positive. It depends only on the point q , not on the vector chosen for it, and replacing p by $e^{i\alpha} p$ replaces $\tilde{q}$ by $e^{i\alpha} \tilde{q}$. Hence $[(1 - u)p + u\tilde{q}]$ depends only on the two points and on u . For an aligned pair, the points $[(1 - u)p + uq]$, $u \in [0, 1]$, trace the minimizing geodesic from p to q , since the real plane spanned by p and q is orthogonal to the fibers of $\mathbb{C}^3 \setminus \{0\} \rightarrow \mathbb{CP}^2$. We call u the *linear parameter* of the geodesic. The geodesic midpoint is its point $u = \frac{1}{2}$, $(p + q)/\|p + q\|$. More generally, we call the normalized sum of phase-aligned representatives (aligned to the first point) the *barycenter* of a set of nearby points; we use it only to place the apices of the closing cones (Section 6).

The affine chart $U_k = \{p_k \neq 0\}$ has coordinates p_i/p_k , $i \neq k$. For a simplex we use its *best-conditioned chart*, the k that maximizes $\min_v |p_k(v)|$ over its vertices v . In this chart all its vertices are far from the line at infinity $\{p_k = 0\}$.

Let C be a smooth curve of degree n . By Wirtinger’s theorem, the area of C in the Fubini–Study metric is n times the area of a projective line [23]. With the distance above, a projective line is a round sphere of radius $\frac{1}{2}$, so $\text{area}(C) = n\pi$. This gives an exact global quantity to compare with the area of a computed triangulation: we use it to measure accuracy, and coverage in Section 7.7.

3.2 Maubach bisection

A cell of a d -mesh is an ordered simplex $\sigma = \langle v_0, \dots, v_d \rangle$ with a level $\text{level}(\sigma) \geq 0$. Maubach’s rule [31] sets $\text{type}(\sigma) = d - (\text{level}(\sigma) \bmod d)$, bisects the edge $v_0 v_{\text{type}(\sigma)}$ at a new vertex v , and replaces σ by the two children

$$\langle v_0, \dots, v_{l-1}, v, v_{l+1}, \dots, v_d \rangle, \quad \langle v_1, \dots, v_l, v, v_{l+1}, \dots, v_d \rangle, \quad l = \text{type}(\sigma),$$

both of level $\text{level}(\sigma) + 1$. To keep the mesh conforming, a cell is bisected only together with every other cell of the same level that contains its split edge, after bisecting recursively any coarser cell containing that edge. This terminates, and produces conforming meshes, when the initial mesh is *compatible* [31, 43, 42]. We use the strongest form of compatibility. An initial mesh is *strongly compatible* if, whenever two cells share a facet, the facet omits the vertex at the same position i in both, and the remaining vertices appear in the same order.

A d -complex is *balanced* if its vertices admit a coloring $\mathbf{c} : V \rightarrow \{0, \dots, d\}$ that is injective on every cell.

Proposition 3.1 ([43], [15, §3.1]). *A conforming d -mesh admits vertex orders that make it strongly compatible if and only if it is balanced. The order of each cell is then the order of the colors of its vertices.*

Indeed, if cells are ordered by color, the common facet of two adjacent cells carries every color except the missing one, which occupies the same position in both. Strong compatibility is a statement about *ordered* facets: matching only the position of the omitted vertex, without the order of the rest, is not enough.

4 An adaptive mesh of $\mathbb{CP}^2$

4.1 The seed

$\mathbb{CP}^2$ has two classical small triangulations. Kühnel’s $\mathbb{CP}_9^2$ [29] has 9 vertices and is vertex-minimal. Gaifullin’s X [21] has 15 vertices, f -vector $(15, 90, 240, 270, 108)$, and automorphism group $S_4 \times S_3$, which acts by Fubini–Study isometries in Gaifullin’s explicit realization. Gaifullin shows that X is balanced: color the three vertices of $V_4 \setminus \{e\}$ with 0 and the vertices (a, b) with a , for $a = 1, \dots, 4$. He also shows that X is vertex-minimal among balanced triangulations of $\mathbb{CP}^2$. By Proposition 3.1, X ordered by its coloring is a strongly compatible seed. Kühnel’s $\mathbb{CP}_9^2$ is not: it is 3-neighborly, so its graph is K_9 , which is not 5-colorable.

Remark 4.1. We verified these facts independently on the facet list of X (with `simpcomp` [19]):

- every vertex has exactly two non-neighbors, namely the other members of its color class;
- all 240 triangles have degree 4 or 6;
- in the color order, all 270 shared facets omit the vertex at the same position on both sides.

The 108 seed cells are also geometrically uniform: the max/min Fubini–Study edge ratio is 1.4636 in every cell, as expected from the isometric symmetry. The barycentric subdivision of $\mathbb{CP}_9^2$ is balanced too, but in Kühnel’s standard realization several flag barycenters coincide with existing vertices, which produces degenerate cells.

Geometry. We use Gaifullin’s coordinates [21], moved by a random unitary change of basis of $\mathbb{C}^3$ to avoid special alignments between the curve and the seed. Each new vertex is placed at the Fubini–Study geodesic midpoint of the split edge, after aligning the phases of the two homogeneous representatives. The coordinates of a cell’s vertices are not stored: they are recomputed from the seed along the cell’s path in the hierarchy, deterministically, so every cell sees the same point for the same vertex.

4.2 Pointerless codes on balanced seeds

Why codes. An explicit mesh stores each simplex with references to its faces and cofaces. The number of faces per cell grows quickly with the dimension. In the Freudenthal–Kuhn triangulation of $\mathbb{R}^d$, which is the combinatorial type of uniform Maubach refinements of the cube, a k -simplex is determined up to translation by an ordered sequence of k disjoint nonempty subsets of $\{1, \dots, d\}$. There are therefore $k! S(d+1, k+1)/d!$ of them per d -cell, where S denotes Stirling numbers of the second kind. A full incidence structure storing only the boundary references ($k+1$ integers per k -simplex) already needs:

d	2	3	4	5	6	7	8
simplices of dim. 1.. d per cell	2.5	4.2	6.2	9.0	13.0	18.8	27.1
boundary references per cell	6.0	12.3	22.5	39.0	65.7	108.3	175.8
LPT code bits (40 levels)	49	52	55	59	62	66	70

A *compact* explicit structure stores cells only, each with its $d+1$ vertex identifiers and $d+1$ neighbor identifiers, i.e. $8(d+1)$ bytes per cell with 32-bit integers (40 bytes for $d=4$) plus bookkeeping for levels and the hierarchy. An LPT code needs about $L + \log_2(2^d d!) + O(\log L)$ bits for L levels, whatever the dimension. Section 7.6 compares memory in practice.

LPT codes. Atalay and Mount [4] encode the Maubach hierarchy of a Kuhn-triangulated cube $[-1, 1]^d$. Its reference root simplex is

$$S_\emptyset = \{x \in \mathbb{R}^d : 1 \geq x_1 \geq x_2 \geq \dots \geq x_d \geq -1\},$$

whose facet H_0 lies on $x_1 = 1$, facet H_j ($0 < j < d$) on $x_j = x_{j+1}$, and facet H_d on $x_d = -1$. A descendant is encoded by its *LPT code* (ℓ, Π, Φ) :

- ℓ is the level modulo d ;
- $\Pi = (\pi_1, \dots, \pi_d)$ is a signed permutation;
- Φ is the orthant list. After every d levels, one bit is appended to each coordinate. We read these bits as integers $o_1, \dots, o_d \in [0, 2^r)$, where r is the number of completed rounds and the oldest bit is the most significant.

Their Theorem 1 gives the vertices explicitly. With $T_j = 2^r - 1 - 2o_j$, the k -th vertex is

$$x^{(k)} = \frac{T + y^{(k)}}{2^r}, \quad y_{|\pi_j|}^{(k)} = \text{sgn}(\pi_j) v_{kj}, \quad v_{kj} = \begin{cases} 1, & k \geq j, \\ 0, & k < \min(j, \ell), \\ -1, & \ell \leq k < j, \end{cases} \quad (1)$$

for $k = 0, \dots, d$ and $j = 1, \dots, d$. Here $y^{(k)} \in \{-1, 0, 1\}^d$ depends only on (ℓ, Π, k) , while T depends only on the orthant list. Their Theorem 2 computes the code of a facet neighbor by table lookup. It describes neighbors in the infinite Kuhn arrangement: in the cube, a neighbor outside one root simplex is another root simplex of the same cube. Our implementation descends from the one we wrote for the Adaptive Voids algorithm [18], where LPT codes over the six Kuhn tetrahedra of a cube drive an adaptive subdivision of a solid.

Balanced seeds. Let K_0 be a balanced d -dimensional seed with N_0 cells, each ordered by color, and identify every seed cell $\tau = \langle \tau[0], \dots, \tau[d] \rangle$ with $S_\emptyset$ by sending the k -th vertex of $S_\emptyset$ to $\tau[k]$. A cell of the refined seed is a pair (s, κ) : a seed index s and an LPT code κ of the reference hierarchy.

We work *combinatorially*: a vertex of the hierarchy of τ_s is identified with its barycentric coordinates $w \in \Delta^d$ with respect to τ_s . These are dyadic rationals, and by (1) the coordinates $w(\kappa, m)$ of the m -th vertex of (s, κ) depend on κ only. A vertex on a facet of τ_s is also a vertex of the neighboring seed cell, via the identification of the common facet. A geometric realization is applied afterwards: affine, or by Fubini–Study midpoints. Since it is defined edge by edge from the endpoints of the split edge, it respects these identifications.

Proposition 4.2 (seed-boundary test). *Facet ∂_i of the cell with code $\kappa = (\ell, \Pi, \Phi)$ lies on the facet H_j of its seed cell if and only if:*

- (i) *the linear part of the equation of H_j is constant on $\{y^{(k)} : k \neq i\}$, i.e. $y_1^{(k)} \equiv 1$ ($j = 0$), $y_{j+1}^{(k)} - y_j^{(k)} \equiv 2\delta$ with $\delta \in \{-1, 0, 1\}$ ($0 < j < d$), or $y_d^{(k)} \equiv -1$ ($j = d$); and*
- (ii) *$o_1 = 0$ ($j = 0$), $o_{j+1} - o_j = \delta$ ($0 < j < d$), or $o_d = 2^r - 1$ ($j = d$).*

At most one j satisfies (i), which depends only on (ℓ, Π, i) .

Proof. Substitute (1) into the equation of H_j . For $0 < j < d$, it reads $T_j + y_j^{(k)} = T_{j+1} + y_{j+1}^{(k)}$ for all $k \neq i$. Since T does not depend on k , this forces $y_{j+1}^{(k)} - y_j^{(k)}$ to be constant, equal to $T_j - T_{j+1} = 2(o_{j+1} - o_j)$. For $j = 0$, the equation $T_1 + y_1^{(k)} = 2^r$ gives $2o_1 = y_1^{(k)} - 1$; since $o_1 \geq 0$ and $y_1^{(k)} \leq 1$, this means $o_1 = 0$ and $y_1^{(k)} = 1$. The case $j = d$ is symmetric. At most one j qualifies, because the d vertices of a facet span a hyperplane with a unique normal direction, and the facet normals of a simplex are pairwise non-parallel. $\square$

Condition (i) is precomputed in a table indexed by (ℓ, Π, i) that stores j and δ , or “none”. For $d = 4$ the table has $4 \cdot 384 \cdot 5 = 7680$ one-byte entries. The test is then a single lookup followed by a single integer comparison.

Proposition 4.3 (seed crossing). *Suppose facet ∂_i of $c = (s, \kappa)$ lies on the facet $\partial_j \tau_s$ of its seed cell, and let s' be the seed cell sharing that facet. Then the neighbor of c across $\partial_i c$ is $c' = (s', \kappa)$, the same code in the other seed cell, and $\partial_i c' = \partial_i c$ as ordered simplices.*

Proof. Since the seed is ordered by color, $\partial_j \tau_s = \partial_j \tau_{s'}$ as ordered simplices, i.e. $\tau_s[p] = \tau_{s'}[p]$ for every $p \neq j$. For $m \neq i$, the m -th vertex of c lies on $\partial_j \tau_s$, so its coordinates satisfy $w_j(\kappa, m) = 0$. The m -th vertex of c' has the same coordinates with respect to $\tau_{s'}$, hence the same position on the common facet. It is therefore the same vertex of the refined seed. So c' contains $\partial_i c$, in the same positions, has the same level as c , and lies in the other seed cell. Uniform refinements of a strongly compatible mesh are conforming [31, 43], so exactly two cells of that level contain $\partial_i c$. Hence c' is the neighbor. $\square$

This is where balance, rather than any compatible ordering, is used: without it the two identifications would differ on the common facet by a permutation, and the code would have to be transformed.

Neighbor finding. To find the neighbor of $c = (s, \kappa)$ across $\partial_i c$:

1. *Sibling facet.* If $\partial_i c$ is the facet created by the last bisection ($i = d$ for a 0-child, $i = \text{type}$ for a 1-child, in the vertex numbering of the codes), return the other child of the parent.
2. *Boundary test.* Otherwise, look up (j, δ) for (ℓ, Π, i) and check the orthant condition of Proposition 4.2.

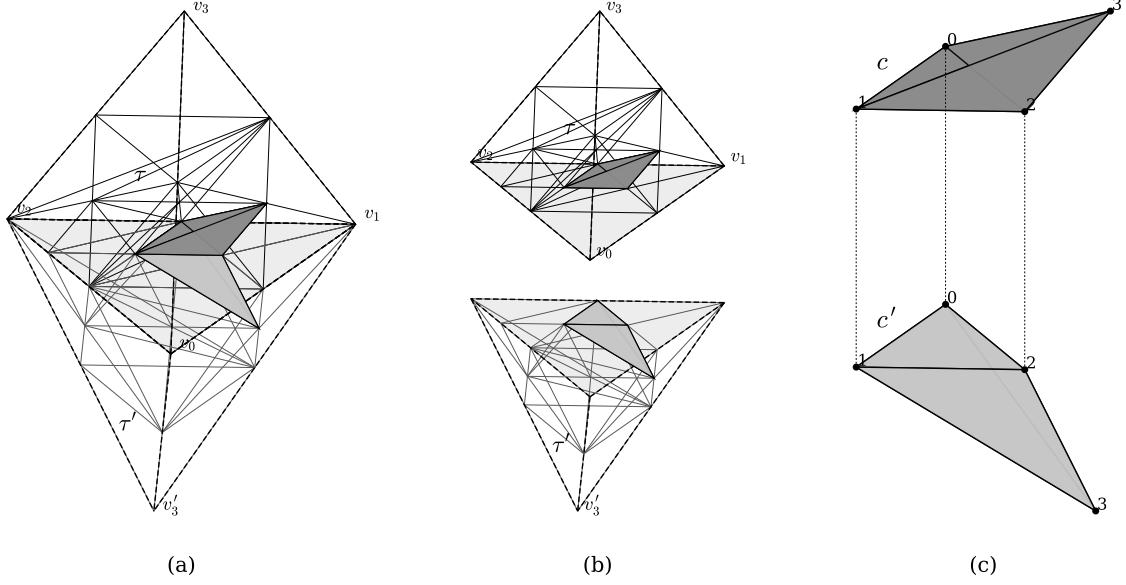


Figure 1: Propositions 4.2 and 4.3 in dimension 3. (a) Two color-ordered seed tetrahedra τ and τ' , glued along their common facet $\langle v_0, v_1, v_2 \rangle$ (light gray), after adaptive Maubach bisection towards a point just above that facet. The conforming closure propagates across the seed boundary. Dashed lines are seed edges. (b) The same mesh, exploded along the normal of the common facet: both seed cells induce the same subdivision on it. (c) The leaf $c \subset \tau$, whose facet $\partial_3 c$ lies on the seed boundary, and its neighbor $c' \subset \tau'$. Both have the same bisection code (path 010010 from their roots), and the shared facet occupies local positions 0, 1, 2 in both. The figure is generated by a script that asserts both facts.

3. *Interior facet.* If the test fails, return (s, κ') with κ' given by Atalay and Mount’s Theorem 2.
4. *Seed crossing.* Otherwise return $(\text{adj}[s][j], \kappa)$ (Proposition 4.3), where adj is the $N_0 \times (d+1)$ seed adjacency table. Root cells use this rule directly, with $j = i$.

Rule 2 is indispensable. Across a seed-boundary facet, Atalay and Mount’s formula returns a cell of the reflected reference simplex, i.e. one with negative barycentric weights. Such crossings are frequent: about 24% of the non-sibling facets sampled at realistic depths lie on the seed boundary. Figure 1 illustrates rules 2 and 4. Here 0-child and 1-child refer to Atalay and Mount’s numbering, and $\text{type} = \ell$ with $\ell = 0$ read as d .

Verification. We checked every returned neighbor *exactly* on Gaifullin’s seed ($d = 4$). Both cells were written as integer barycentric coordinates over the 15 global vertices, with common denominator 2^{11} , and we checked that the returned cell is a different cell of the same level containing the queried facet. The test covered 10^6 random cells over the whole representable depth range and five million queries: 977 070 sibling, 3 317 979 interior and 704 951 seed-crossing. None failed, and away from sibling facets the facet always sat at the same local index on both sides. An earlier implementation computed the boundary test from the full matrix of barycentric weights ($O(d^2 + rd)$ integer operations) and resolved crossings by floating-point point location. The table-based test agrees with it on 7.8 million facets, and same-code crossing agrees on 1.4 million crossings. The new rules take 4.3 ns and 5 ns per query, against 79 ns and 211 ns.

The leaf set. The mesh is the set of codes of the current leaves, stored in an open-addressing hash table: one 64-bit slot per leaf, linear probing, and backward-shift deletion, which avoids

tombstones under the erase-two-insert churn of bisection. For Gaifullin’s seed a code fits in one word: 7 bits of seed index, 2 bits for ℓ , 9 for Π , 4 for r and 4×10 for the orthant list, i.e. 62 bits and a depth of up to 43. Two operations are built on the table:

- `compat_bisect`(c) is Maubach’s conforming bisection on codes. It bisects coarser neighbors recursively before bisecting c , so the mesh stays graded: no facet separates cells more than one level apart;
- `neighbor_leaf`(c, i) walks up the ancestor chain of the combinatorial neighbor to the current leaf, or reports that the region has been refined further.

An exhaustive graded-invariant test, with about 40 000 cascading bisections, 4 million leaves and 11.6 million facet checks, found no violation. Vertex identifiers are not part of the code. When needed (for geometry), a new vertex receives an integer identifier, keyed by the identifiers of the endpoints of the split edge in a separate hash table; vertices are about $1/d!$ as numerous as cells.

Limitations. Enumerating the several finer leaves that border a coarser facet is not implemented; such queries are reported, never guessed. Seeds that are compatible but not balanced would need a permutation correction in Proposition 4.3. For the initialization of [15] with $N > d$ colors, a cell is a face of a virtual N -simplex. Refinements of the virtual extension do not restrict to refinements of the mesh [15, Rem. 12], so it is unclear how to encode them.

5 Extraction and continuation

5.1 Realizing the faces

The mesh is combinatorial; extraction needs every triangular 2-face as an actual surface in $\mathbb{CP}^2$, on which $F = 0$ can be solved. The obvious choice, the flat triangle spanned by the three vertices in an affine chart, depends on the chart. A face shared by many cells is solved once, so it can use only one chart, and the natural choice is the face’s own best-conditioned chart. But then two faces that share an edge and use different charts see that edge as two different curves (Figure 2a). The four faces of a tetrahedral facet no longer form a closed surface, and the curve can cross the resulting seams an odd number of times.

In our first implementation this was the dominant source of failures (Section 7.1).

We realize every face without charts. Let A, B, C be its vertices, sorted by global identifier, with their unit representatives. The face is the *geodesic cone* from A over the geodesic BC :

$$q(t) = (1 - t)B + t\tilde{C}, \quad v(s, t) = (1 - s)A + s\tilde{q}(t), \quad (s, t) \in [0, 1]^2,$$

where $\tilde{C}$ is C aligned to B and $\tilde{q}(t)$ is $q(t)$ aligned to A (Section 3). The first alignment makes q the geodesic BC ; the second makes each segment $s \mapsto v(s, t)$ the geodesic from A to $q(t)$. By the invariance of alignment, the point $[v(s, t)]$ does not depend on the representatives chosen for A, B, C . On the boundary, $t = 0$, $t = 1$ and $s = 1$ give the geodesics AB , AC and BC , each traversed by its linear parameter from the lower identifier to the higher one. An edge is therefore the same curve, with the same parameter, in every face that contains it (Figure 2b), and the Fubini–Study midpoint used for bisection lies on it. We use

$$(\lambda_A, \lambda_B, \lambda_C) = (1 - s, s(1 - t), st)$$

as barycentric coordinates on the face; on each edge they reduce to the linear parameter.

Proposition 5.1. *Let T be a tetrahedral facet of the mesh whose four realized faces lie in a geodesic ball of radius less than $\pi/2$. If C meets these faces transversally, at points interior to the faces, then it meets them in an even number of points.*

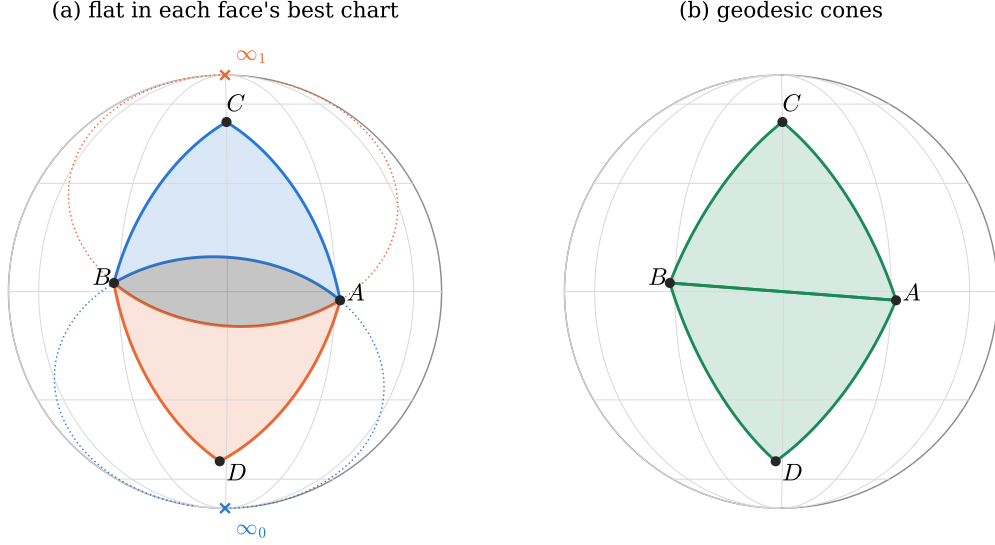


Figure 2: Two realizations of two triangles ABC and ABD that share the edge AB , drawn on a projective line $\mathbb{CP}^1 \subset \mathbb{CP}^2$, which in the Fubini–Study metric is a round sphere of radius $\frac{1}{2}$; a triangle with its vertices on that line lies on it entirely. (a) Each triangle is flat in its own best-conditioned chart, chosen by the rule of the implementation: ABC in the chart $p_0 \neq 0$, ABD in the chart $p_1 \neq 0$. A segment that is flat in a chart is an arc of a circle through that chart’s point at infinity (∞_0, ∞_1 ; dotted), so the two versions of AB differ and enclose a seam (gray). Here $|AB| = 0.55$, comparable to the cells of depths 12–14, and the two versions are up to 0.16 apart. (b) Geodesic cones: both triangles are bounded by the same great-circle arc AB . Generated by `fig_faces.py`.

Proof. The four faces glue along their common edges into a map $\phi : S^2 \rightarrow \mathbb{CP}^2$. A geodesic ball of radius less than $\pi/2$ around p lies in the affine chart $\{x : \langle x, p \rangle \neq 0\} \cong \mathbb{C}^2$, which is contractible, so ϕ extends to a map of the 3-ball. Its image is a singular 3-chain D with $\partial D = \phi(S^2)$. Since C is closed, its intersection number with a boundary vanishes, so $\#(C \cap \phi(S^2)) \equiv 0 \pmod{2}$. $\square$

With chart-flat faces the four faces do not glue, ϕ is not defined, and odd counts occur. In practice the hypothesis on the ball holds for all cells after the first few levels.

5.2 Per-cell extraction

For a leaf c , we compute the crossings of C with the 10 triangular 2-faces of c , join them into arcs on its 5 tetrahedral facets, and assemble the arcs into closed polygons that approximate $C \cap c$. Crossings are cached by the vertex identifiers of their face, so adjacent cells see identical crossings.

Why 2-faces: a k -simplex and the real surface C meet, generically, in a set of dimension $k + 2 - 4$. So C misses the edges and meets each triangular face in isolated points; it meets each tetrahedral facet in arcs whose endpoints lie on the four triangles of that facet; and it meets the cell in a surface piece whose boundary is a closed curve through the facets. The extraction follows these dimensions upward.

- *Crossings.* On a face, $F(v(s, t)) = 0$ is a system of two real equations in (s, t) ; since F is homogeneous, v need not be normalized. Newton’s method uses the analytic Jacobian of $(s, t) \mapsto F(v(s, t))$, and is started from the zero of the linear interpolant of F at the aligned vertices and from four fixed points of the face. Distinct roots in $[0, 1]^2$ are kept, up to two per face.

Algorithm 1 Continuation to target depth D

```
1:  $s \leftarrow$  the first seed cell whose extraction has at least one arc
2:  $Q \leftarrow (s)$ ;  $visited \leftarrow \emptyset$ 
3: while  $Q \neq \emptyset$  do
4:    $c \leftarrow \text{pop}(Q)$ 
5:   if  $c$  is not a current leaf or  $c \in visited$  then continue
6:   end if
7:   add  $c$  to  $visited$ ; extract  $c$ 
8:   if  $c$  has no arc then continue  $\triangleright$  the curve does not reach  $c$ 
9:   end if
10:  if  $\text{level}(c) < D$  or  $c$  is near-tangent (Section 5.4) then
11:     $\text{compat\_bisect}(c)$ ; push the new leaves onto  $Q$ 
12:  else
13:    for  $i = 0, \dots, d$  do
14:      if  $\text{neighbor\_leaf}(c, i)$  finds a leaf  $c' \notin visited$  then push  $c'$  onto  $Q$ 
15:    end for
16:  end if
17: end while
18: extract every leaf that meets  $C$ ; output the polygons of the good cells
```

- *Arcs.* On each tetrahedral facet, two crossings are joined by an arc. With four crossings, they are paired so as to minimize the total Fubini–Study length of the two arcs. A single crossing (the curve only touches the facet) is ignored, and three or more than four make the facet *irregular*.
- *Polygons.* The arcs form a graph on the crossings of the cell. If every node has degree 2, the graph is a disjoint union of cycles, each with 3 to 5 nodes, and each cycle is output as one polygon.

A crossing on an edge or at a vertex, up to a tolerance, is identified by that edge or vertex, so that all cells containing it agree on its identity. A cell is *bad* when the assembly fails: a node of degree other than 2, an irregular facet, or a cycle with fewer than 3 or more than 5 nodes. A planar piece meets a 4-simplex in a convex polygon with at most one edge per facet, so longer cycles signal strong curvature or a near-tangency. With geodesic faces, the remaining failures occur at near-tangencies (Section 7.2). A bad cell contributes no polygon and leaves a hole in the output.

5.3 Continuation

Starting from one seed cell that meets C , a frontier of cells is processed as follows:

- a cell that meets C and is below the target depth is bisected with compat_bisect , and its new leaves join the frontier;
- a cell at the target depth is extracted, and its neighbors across the five facets, found by neighbor_leaf , join the frontier.

Moving through the mesh thus uses only the combinatorial operations of Section 4.2. The continuation reaches every cell whose 2-faces are crossed by a connected curve. A small closed component of $C \cap c$ that meets no 2-face of c would be missed, and a reducible curve needs one starting cell per component.

Algorithm 1 summarizes the procedure.

5.4 Refinement near tangencies

Two optional mechanisms refine further where extraction is likely to fail:

- a *tangency threshold* on a Cauchy–Riemann transversality score computed at each crossing;
- *repair rounds* that re-bisect bad cells.

The *transversality score* of a crossing $x = [v(s, t)]$ on a face is

$$\tau(x) = \frac{|\operatorname{Im}(\bar{w}_1 w_2)|}{|e_1||e_2||\nabla F(v)|^2}, \quad w_k = \nabla F(v) \cdot e_k,$$

where e_1, e_2 are $\partial_s v, \partial_t v$ projected orthogonally to v , and $\nabla F = (\partial_X F, \partial_Y F, \partial_Z F)$. Since F is holomorphic and $\nabla F(v) \cdot v = nF(v) = 0$, w_k is the derivative of F along the face in the direction e_k , and $\operatorname{Im}(\bar{w}_1 w_2)$ is the Jacobian determinant of $(\operatorname{Re} F, \operatorname{Im} F)$ restricted to the face. By Cauchy–Schwarz, $\tau \in [0, 1]$, with $\tau = 0$ exactly when the plane of the face is not transverse to C at x . A cell’s score is the minimum over its crossings. With a threshold $t > 0$, a cell at the target depth whose score is below t is bisected further, up to a fixed number of extra levels (four by default). A repair round bisects every bad leaf once, within its own budget of extra levels (also four by default), and extracts every leaf again.

6 Topological verification and closing

The output is not certified. Its topology can nevertheless be checked, directly on the extracted 2-complex in $\mathbb{CP}^2$, independently of any projection. Vertices are identified by the mesh vertex, edge or 2-face on which the crossing lies. Bad cells leave holes. We split each *pinched* vertex into one copy per component of its link. In all our runs these vertices lie on hole boundaries, where a hole touches itself or two holes meet. If the holes are discs, then $g = (2c - b - \chi)/2$, where c is the number of components and b the number of boundary loops. A closing step then turns the complex into a closed surface:

- it removes every polygon on an edge shared by more than two polygons. Each cell passes its own checks, but the cells do not agree there; we have seen this only at near-tangencies, where a 2-face carries two close crossings and several cells join them;
- it keeps the main component;
- it caps each hole by a cone from a new apex, placed at the loop’s Fubini–Study barycenter and moved onto C by Newton’s method;
- it checks that the result is a connected closed surface with $\chi = 2 - 2g$.

These are necessary conditions. A missing handle and a spurious one would cancel in χ , and χ says nothing about the embedding. They are nevertheless a useful sanity check for an uncertified method.

7 Experiments

All runs used one core of an AMD Ryzen 5 5625U laptop with 7.2 GB of RAM, and code compiled with GCC 16. The curves are smooth:

name	equation	genus
conic	$XY - Z^2$	0
elliptic	$X^2Z - Y^3 + YZ^2$	1
Fermat cubic	$X^3 + Y^3 + Z^3$	1
Fermat quartic	$X^4 + Y^4 + Z^4$	3

Unless stated otherwise, runs realize the faces as geodesic cones (Section 5.1) and use one fixed random rotation of the seed (generator seed 12345). The *base mode* is the continuation of Algorithm 1 at a fixed target depth, without the tangency threshold or the repair rounds of Section 5.4; these two mechanisms are always reported as additions to it. The scripts that produce every table and figure are listed in the code availability statement.

7.1 Chart-flat faces versus geodesic cones

Our first implementation realized each face as a flat triangle in its own best-conditioned chart (Section 5.1). We found the effect of that choice with a per-cell diagnostic, which records whether all ten faces of a cell were solved in the cell’s own best-conditioned chart. On the elliptic curve at depth 12, the bad-cell rate was 0.21% among cells whose faces all used the cell’s chart and 21% among the others; at depth 14, 0.11% against 14%, and 0.10% against 18% for the Fermat quartic. More than 97% of the bad cells were of the second kind. Geodesic cones remove the seams, and with them most failures (depth 14, base mode):

curve	chart-flat faces		geodesic cones	
	bad cells	rate	bad cells	rate
conic	350	1.84%	2	0.010%
elliptic	887	3.14%	34	0.120%
Fermat cubic	2 663	9.91%	22	0.080%
Fermat quartic	4 347	11.6%	53	0.136%

Facets with a single crossing, which Proposition 5.1 rules out for closed facet boundaries, drop from 672–8 463 per run to 0–8. Geodesic cones cost about 20–27% more running time, spent in the evaluation of the cone parametrization.

7.2 Bad-cell rate

Figure 3 shows the bad-cell rate for four curves of degree 2 to 4. The number of bad cells stays nearly constant with depth, between 2 and 59 per run, while the number of cells that meet the curve doubles roughly every two levels; the rate therefore halves roughly every two levels: it is below 0.14% at depth 14 and below 0.08% at depth 16 for all four curves. The tangency threshold combined with repair rounds lowers it further, by a factor of up to 6, except for the conic at depths 14 and 16, where only two to five cells fail in either mode. For the elliptic curve, the first row of each depth isolates the effect of the faces (chart-flat faces against geodesic cones, both in base mode), and the following rows add the two refinement mechanisms to geodesic cones:

depth	faces	refinement	leaves	good cells	bad cells	bad rate
14	chart-flat	base	327 370	27 397	887	3.136%
14	geodesic cones	base	326 260	28 278	34	0.120%
14	geodesic cones	+ repair rounds (2)	330 616	28 568	13	0.045%
14	geodesic cones	+ tangency threshold 0.05	505 654	37 490	7	0.019%
14	geodesic cones	+ both	505 654	37 490	7	0.019%
20	chart-flat	base	4 620 102	222 981	1 348	0.601%
20	geodesic cones	base	4 618 522	224 425	14	0.006%
20	geodesic cones	+ repair rounds (2)	4 620 730	224 508	8	0.004%
20	geodesic cones	+ tangency threshold 0.05	6 187 942	305 531	20	0.007%
20	geodesic cones	+ both	6 188 286	305 556	18	0.006%

At depth 20 only 14 of 224 439 cells fail in base mode. Repair rounds still halve that number, but the tangency threshold no longer helps: it adds a third more leaves, and the rate stays near 0.006%.

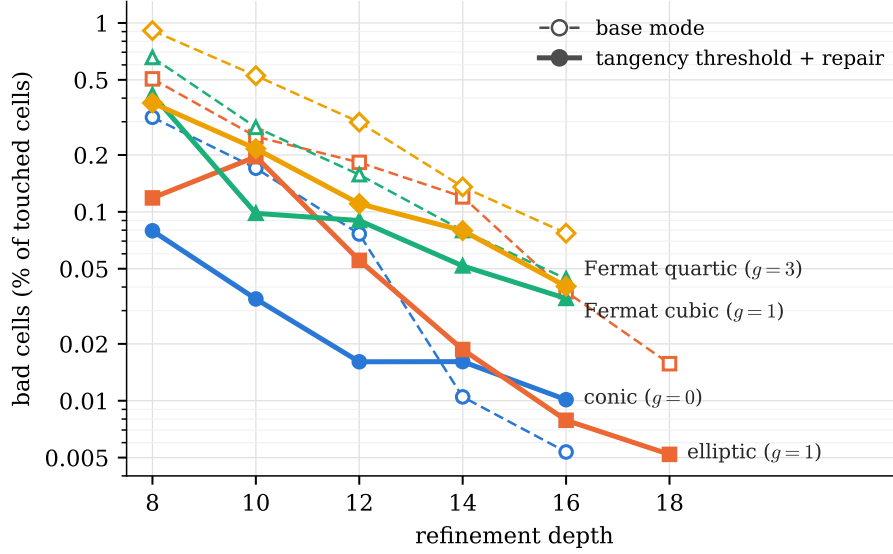


Figure 3: Percentage of touched cells whose extraction fails, against the refinement depth. Dashed lines with open markers: base mode. Solid lines: with tangency threshold 0.05 and two repair rounds.

Table 1: Robustness across five random rotations of the seed (depth 14, tangency threshold 0.05, two repair rounds). “Closed” reports χ of the closed surface, which in all runs has no boundary and one component.

curve	bad cells (min–max)	genus estimate	closed χ	expected
elliptic ($g = 1$)	0.015–0.066%	1 in 5/5	0 in 5/5	0
Fermat quartic ($g = 3$)	0.030–0.061%	3 in 5/5	–4 in 5/5	–4

Causes of failure. We recorded the reason for each failure at depth 14, in base mode, on the four curves (114016 good cells and 111 bad ones):

- 88% of the bad cells fail on a cycle with more than five nodes, 10% on an irregular facet and 2% on node degree;
- the transversality score separates the two populations: its median is 0.29 on good cells on every curve, and between 0.014 and 0.037 on bad cells;
- 79% of the bad cells have a score below 0.05.

With geodesic faces, then, the remaining failures are near-tangencies of C with faces of the mesh, which is what the tangency threshold and the repair rounds target.

7.3 Topology

Table 1 repeats the construction for five random rotations of the seed. In all ten runs the genus estimate is exact, and the closed surface passes every check. The bad-cell rate varies with the rotation by a factor of up to 4, but stays below 0.07% (with chart-flat faces it was 1.0–1.9% and 6.0–7.2%).

Genus of the raw extraction. With the notation of Section 6, tangency threshold 0.05 and two repair rounds:

curve	depth	bad cells	pinched	c	b	χ	g est.	g
conic $XY - Z^2$	14	4 (0.016%)	0	1	2	0	0	0
elliptic	14	7 (0.019%)	0	1	1	-1	1	1
elliptic, Figure 4(a)	18	8 (0.005%)	0	1	3	-3	1	1
Fermat cubic	14	20 (0.052%)	0	1	6	-6	1	1
Fermat quartic, Figure 4(b)	17	67 (0.042%)	1	1	17	-21	3	3

The raw extraction is already connected in all five runs, and has a single pinched vertex in total, against 3 to 36 per run with chart-flat faces. In every case the closed surface is connected, has no boundary, and has $\chi = 2 - 2g$, and Newton’s method moved every cone apex onto the curve.

7.4 Accuracy

In the five runs above, $|F|$ at the extracted vertices, with unit-norm homogeneous representatives, has median below 1.5×10^{-16} , 99th percentile below 7.7×10^{-16} and maximum below 7.2×10^{-14} . These are the crossings of C with faces of the mesh, so they lie on the curve to working precision.

Wirtinger’s theorem gives a global check on the polygons between them. We measure the Fubini–Study area of the closed surface, fanning each polygon into triangles measured in the horizontal tangent space at one vertex, and compare it with $n\pi$:

curve	depth	area/ $n\pi$, raw	area/ $n\pi$, closed
conic	14	0.99989	1.00003
elliptic	14	1.00111	1.00114
elliptic	18	1.00043	1.00046
Fermat cubic	14	1.00078	1.00142
Fermat quartic	17	1.00079	1.00128

The area is within 0.15% of the exact value in every run. In base mode, the error decreases steadily with depth (`area_sweep.sh`): for the closed surface it falls from 0.40–0.92% at depth 10 to 0.06–0.13% at depth 18 on the elliptic curve and the Fermat cubic and quartic, and stays below 0.03% for the conic. The closed surface always slightly overestimates the area. The raw extraction lies below it by the area of the holes, which can bring it slightly under $n\pi$; both converge to $n\pi$.

7.5 Higher degree

To see how the method scales with the degree, we added the Fermat curves of degree 5 and 6 and, for $n = 3, \dots, 6$, a random curve from the Kostlan ensemble: the coefficient of $X^i Y^j Z^k$ is a complex Gaussian of variance $n!/(i! j! k!)$. This distribution is invariant under unitary changes of coordinates, so the curve has no special position relative to the seed, and the curve is smooth with probability 1. At depth 16 (`degree_sweep.sh`):

curve	n	g	meeting C	bad rate		area/ $n\pi$	time (s)
				base	combined		
Fermat	3	1	54 272	0.044%	0.035%	1.00127	27
	4	3	76 531	0.077%	0.040%	1.00193	36
	5	6	93 694	0.084%	0.040%	1.00257	42
	6	10	114 605	0.079%	0.036%	1.00321	51
Kostlan	3	1	51 736	0.070%	0.026%	1.00081	29
	4	3	70 118	0.086%	0.025%	1.00150	46
	5	6	94 033	0.088%	0.044%	1.00233	76
	6	10	103 462	0.100%	0.051%	1.00335	111

“Combined” is tangency threshold 0.05 with two repair rounds; the area is that of the closed surface, the time that of the base mode. Up to degree 6 the method does not degrade:

- the number of cells that meet C grows like the area $n\pi$, with 17–19 thousand cells per unit of degree, so the cost of a curve is proportional to its degree at fixed depth;
- the bad-cell rate grows only from 0.04 to 0.10% in base mode, and stays at 0.025–0.051% with refinement near tangencies;
- in all 48 runs (depths 12, 14 and 16, both modes) the closed surface is connected and has $\chi = 2 - 2g$, up to genus 10; $|F|$ at the extracted vertices is below 1.8×10^{-15} ;
- the closed surface overestimates the area by 0.08 to 0.34%, more for higher degree; the raw extraction is within 0.14%.

Higher degree means more curvature per cell at a given depth, and with it more near-tangencies. On the Fermat sextic, three runs produced an edge shared by four polygons: a 2-face with two crossings 0.035 apart, joined by several cells that each pass their own checks. Removing these polygons before closing (Section 6) gives the right genus in all three. The Kostlan curves are slower than the Fermat curves of the same degree because their polynomials are dense (28 terms for $n = 6$, against 3), and evaluating F dominates the running time.

7.6 Cost

We measured one full incidence structure, VGTL’s `nmt<4>` [34], against the pointerless leaf table of Section 4.2, on uniform refinement of Gaifullin’s seed with no coordinates and no other data. Both hold exactly the same $108 \cdot 2^D$ leaves.

D (leaves)	<code>nmt<4></code>		pointerless	
	cells stored	bytes/leaf	table	bytes/leaf
8 (27 648)	55 188	222	0.5 MB	19.5
12 (442 368)	884 628	222	8.6 MB	19.5

`nmt<4>` keeps every cell ever created, so that coarsening remains possible, together with all faces; this costs 222 bytes per leaf. The pointerless table stores one 64-bit code per leaf at load factor 0.41, i.e. 19.5 bytes per leaf. That is about $11\times$ less than this full incidence structure, and about $2\times$ less than the 40-byte compact structure estimated in Section 4.2. Neither figure includes per-vertex data such as coordinates, which both approaches need when geometry is required.

Continuation from a single seed cell produces exactly the same leaf set as continuation started from all 108 seed cells: we compared the two by an order-independent hash of the leaf codes, for the four curves at depths 8, 12 and 14, and they agree in all twelve cases. Since the curve is connected, nothing is lost by starting from one cell. The mesh is also much smaller than with an earlier criterion, a priority queue driven by the variation of the gradient, which refines cells that do not meet the curve as well: by a factor of 1.2–1.5 at depth 8, 2.0–2.6 at depth 12 and 3.1–4.3 at depth 14.

Running time and memory. On one core, in base mode:

curve	depth	leaves	meeting C	refine (s)	extract (s)	peak (MiB)
elliptic	12	169 304	13 680	2.8	1.1	33
elliptic	14	326 260	28 312	5.6	2.5	60
elliptic	16	952 060	55 594	16.7	8.5	222
elliptic	18	1 698 588	114 607	33.0	16.8	435
elliptic	20	4 618 522	224 439	90.3	56.5	867
Fermat quartic	12	207 250	18 771	3.7	1.6	59
Fermat quartic	14	407 078	39 095	7.5	3.4	112
Fermat quartic	16	1 225 880	76 531	21.9	11.5	224
Fermat quartic	18	2 234 668	156 936	44.4	26.6	438

Time grows roughly linearly with the number of leaves, at 23–32 μ s per leaf over both phases. Refinement dominates, because every cell that continuation visits is extracted once to decide whether the curve reaches it. Peak memory is 190–300 bytes per leaf. Most of it is taken by the caches that make extraction consistent: crossings per face, vertex identifiers and coordinates, and the set of visited cells. The leaf table itself needs 19.5 bytes per leaf. The tangency threshold and repair rounds multiply the running time by 1.4–2.9. With the `nmt<4>` structure, an earlier version of the same pipeline needed 4–5 GB around depth 18; the pointerless mesh needs 867 MiB at depth 20.

7.7 Comparison with an affine chart

The most direct alternative is to mesh $\{F(x, y, 1) = 0\}$ in an affine chart, as Weigle and Banks do on uniform grids of $\mathbb{R}^4$ [44]. We built this baseline from the same code, changing only the geometry. The seed is Kuhn’s triangulation of the box $[-R, R]^4$ in the coordinates $(\operatorname{Re} x, \operatorname{Im} x, \operatorname{Re} y, \operatorname{Im} y)$: 24 cells, colour-ordered, with boundary, which the pointerless mesh accepts once the seed is a parameter. New vertices are affine midpoints, and every face is flat in the one chart, so the faces glue exactly here as well. Continuation, extraction and the topological checks are unchanged. The curve is moved by the same random unitary transformation that moves Gaifullin’s seed in the intrinsic runs; without it, a curve with real coefficients has its real locus inside a union of faces of the axis-aligned box, and extraction degenerates.

To compare the two meshes at equal resolution, we refine every cell that meets C until its Fubini–Study diameter is below h . With $h = 0.2$:

curve	mesh	leaves	meeting C	bad	b	area/ $n\pi$	time (s)
elliptic	$\mathbb{CP}^2$	1 023 146	59 287	16	7	1.000	28
	box, $R = 2$	322 516	19 814	9	5	0.784	11
	box, $R = 4$	562 258	33 166	9	5	0.923	20
	box, $R = 8$	810 518	47 642	12	7	0.981	31
	box, $R = 16$	1 065 792	62 365	12	7	0.994	45
Fermat quartic	$\mathbb{CP}^2$	1 396 928	85 555	68	25	0.999	41
	box, $R = 2$	401 822	25 735	46	14	0.711	18
	box, $R = 4$	753 240	46 011	54	18	0.906	35
	box, $R = 8$	1 143 860	67 737	57	19	0.967	57
	box, $R = 16$	1 531 978	89 821	57	20	0.987	85

Here b counts the boundary loops of the raw extraction: holes left by bad cells and, in the box, the loops where C leaves it. The runs with $h = 0.3$ (in `affine.csv`) show the same pattern:

- *Coverage.* The box misses neighbourhoods of the points of C at infinity. The missing area falls roughly like $1/R^2$, from 22% at $R = 2$ to 0.6% at $R = 16$ for the elliptic curve, and from 29% to 1.3% for the quartic. The intrinsic mesh recovers the whole area, within 0.13% at $h = 0.2$ and 0.26% at $h = 0.3$, the difference being the holes left by bad cells.

- *Cost.* Points far out in the chart are close together in $\mathbb{CP}^2$, so a box large enough to cover most of the curve needs many levels of refinement near its boundary. At $R = 16$ the box already uses 4–10% more leaves than the intrinsic mesh of all of $\mathbb{CP}^2$ at $h = 0.2$, and 33–45% more at $h = 0.3$.
- *Topology.* The box output is a surface with boundary, whose genus can be inferred only when the loops around infinity bound discs, and which cannot be closed by capping holes, since some boundary loops are not holes. The intrinsic output closes into a surface with $\chi = 2 - 2g$.

Bad cells are rare in both meshes. In one chart, flat faces glue just as geodesic faces do in $\mathbb{CP}^2$, which confirms that the gluing of faces, not the use of charts as such, is what matters for extraction. Covering all of $\mathbb{CP}^2$ with charts would take three of them, and their meshes would have to be stitched along the overlaps, where the faces are flat in different charts and no longer glue: precisely the situation that geodesic faces avoid.

7.8 Comparison with $\mathbb{CP}^1 \times \mathbb{CP}^1$

The product compactification $\mathbb{CP}^1 \times \mathbb{CP}^1 \supset \mathbb{C}^2$ is the other natural setting, and it is also the one of our earlier implementation. It is not neutral with respect to the curve. The closure of the elliptic curve $w^2 = z^3 - z$ in $\mathbb{CP}^1 \times \mathbb{CP}^1$ has bidegree $(2, 3)$, so if it were smooth its genus would be $(2 - 1)(3 - 1) = 2$; since the curve has genus 1, the closure is singular. In the coordinates $u = 1/w$, $v = 1/z$ near (∞, ∞) the equation becomes $v^3 = u^2(1 - v^2)$, a cusp $v^3 = u^2$ at the origin. In $\mathbb{CP}^2$ the closure $X^2Z = Y^3 - YZ^2$ is smooth, and the method needs no special care at infinity.

To measure what the cusp costs, we ran the same pipeline on $\mathbb{CP}^1 \times \mathbb{CP}^1$, changing only the seed and the geometry (`riemann_s2s2_glpt`). The seed is the product of two octahedra, triangulated by staircases: 36 vertices and 384 cells. It is balanced: colouring each octahedron by its three antipodal pairs and the vertex (a, b) by $\mathbf{c}(a) + \mathbf{c}(b)$ gives every staircase the colours $0, \dots, 4$ in order. New vertices are Fubini–Study midpoints in each factor, faces are geodesic cones built factor by factor as in Section 5.1, and F is evaluated bihomogeneously, with no chart. With the Fubini–Study metric on each factor, a curve of bidegree (n_w, n_z) has area $\pi(n_w + n_z)$, which plays the role of Wirtinger’s theorem. We compare the elliptic curve with the parabola $w^2 = z$, whose closure $v = u^2$ is smooth at the corner, in base mode; “near” means a cell with a vertex within 0.1 of (∞, ∞) in the product metric:

curve	depth	bad near / near	bad elsewhere	area/ $\pi(n_w+n_z)$	closed χ
elliptic	12	19 / 111	0.092%	1.00044	0
	14	23 / 300	0.058%	1.00024	0
	16	30 / 576	0.042%	1.00012	0
parabola	12	0 / 42	0.065%	0.99947	2
	14	0 / 124	0.021%	0.99971	2
	16	0 / 232	0.014%	0.99987	2

Away from the corner both curves behave as in $\mathbb{CP}^2$. At the corner the parabola has no failure at all, while 5 to 18% of the elliptic curve’s cells there fail, at every depth and in five further rotations of the seed at depth 14. Their number grows slowly with depth while the area they cover shrinks, as expected near a singular point where the curvature of C is unbounded.

The cusp also affects the cells that do not fail. Some of their polygons cut across the cusp instead of following the curve, and they glue with the wrong orientation: without further care the extracted complex is not orientable, and closing it gives $\chi = -1$. A holomorphic curve carries a canonical orientation, and the Kähler form ω of the ambient space restricts to its area form, so a polygon close to the curve has $|\omega|/\text{area}$ close to 1, the value attained exactly on complex lines.

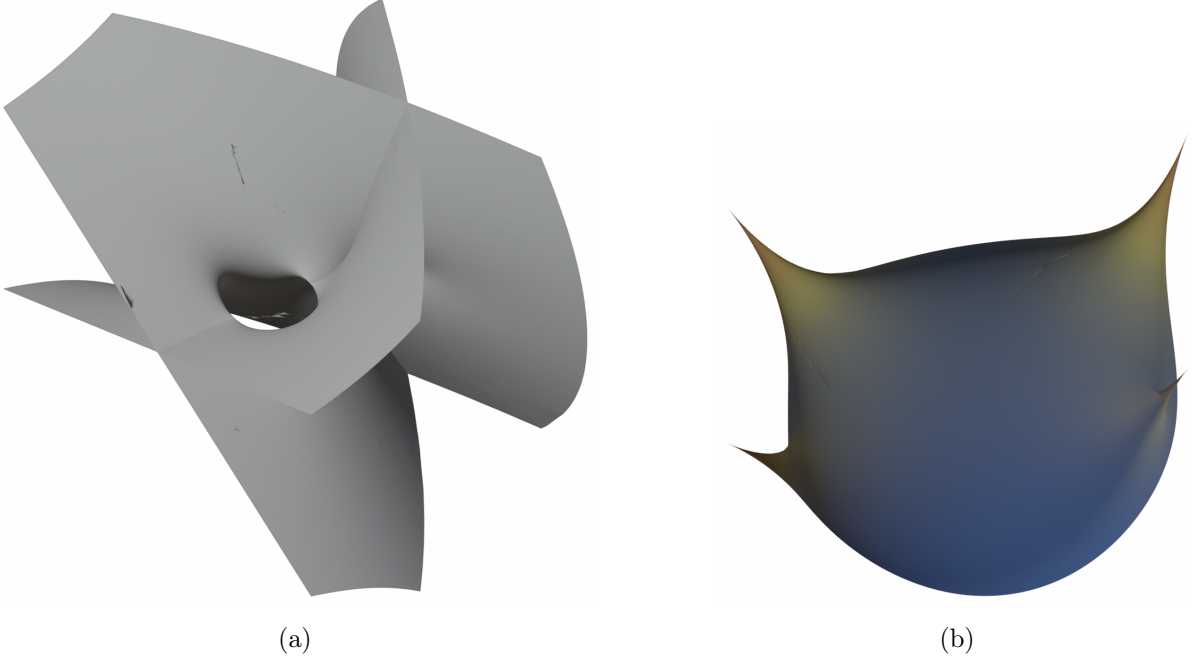


Figure 4: Curves triangulated inside $\mathbb{CP}^2$. (a) The elliptic curve $X^2Z - Y^3 + YZ^2 = 0$, i.e. $w^2 = z^3 - z$ in the chart $Z = 1$ ($w = X$, $z = Y$), displayed as the graph $(\operatorname{Re} z, \operatorname{Im} z, \operatorname{Re} w)$, clipped to $|\operatorname{Re} z|, |\operatorname{Im} z| \leq 2$ and $|\operatorname{Re} w| \leq 3$; depth 18, 8 bad cells (0.005%). (b) The Fermat quartic $X^4 + Y^4 + Z^4 = 0$ (genus 3) under Dutter’s chart-free map $\tilde{\alpha} : \mathbb{CP}^2 \rightarrow \mathbb{R}^3$ [17], colored by distance to the center and drawn semi-transparent; depth 17, 67 bad cells (0.042%). Both surfaces were closed as described in Section 6. The few faint marks are discontinuities of the interpolated normals, at closing cones and at thin polygons of the extraction, not holes.

Treating as bad every cell with a polygon below 0.9 removes 1 to 18 cells of the elliptic curve per run, mostly near the corner, and at most one of the parabola; the closed surface is then an orientable torus in all nine runs of `s2s2_sweep.sh`. The test applies equally in $\mathbb{CP}^2$, where we have not needed it. Our earlier $\mathbb{CP}^1 \times \mathbb{CP}^1$ implementation, with chart-flat faces and a switch to the chart $(1/w, 1/z)$ near the corner, failed at the corner for both curves; with geodesic cones that failure disappears, which isolates the cost of the cusp. In $\mathbb{CP}^2$ the closure is smooth and neither the cusp nor the corner arises.

8 Visualization

The triangulation lives in $\mathbb{CP}^2$; to draw it, we map it to $\mathbb{R}^3$. We use two maps.

- *Flat charts.* A point of the chart U_k with coordinates (a, b) is drawn at $(\operatorname{Re} a, \operatorname{Im} a, \operatorname{Re} b)$, i.e. as the graph of $\operatorname{Re} b$ over the a -plane, clipped to a box. The chart is chosen per polygon, as the best-conditioned chart of its vertices, or fixed for the whole picture. A per-polygon choice keeps every polygon bounded, but adjacent polygons drawn in different charts do not match: the picture is a union of pieces of three graphs. A fixed chart gives one consistent graph, but loses the curve near the line at infinity of that chart. Figure 4(a) uses the fixed chart $Z = 1$.
- *Dutter’s map* [17], after Kranich [28], maps all of $\mathbb{CP}^2$ continuously into a bounded region of $\mathbb{R}^3$, with no chart. It shows the whole closed surface (Figure 4(b)), at the cost of self-intersections in the image.

9 Limitations and conclusion

We triangulate smooth plane curves intrinsically in $\mathbb{CP}^2$ by adaptive Maubach bisection of a balanced seed, Newton crossings on 2-faces, and continuation through a pointerless mesh. Working in $\mathbb{CP}^2$ removes charts, cuts and the special role of infinity. Balance of the seed is what makes both bisection and the pointerless neighbor procedure work on a closed manifold, and the same code runs unchanged on $\mathbb{CP}^1 \times \mathbb{CP}^1$ once its seed and geometry are swapped.

The method has clear limits.

- It is not certified. The bad-cell rate decreases with depth but does not vanish, and the topology is checked only a posteriori, through necessary conditions.
- It assumes a smooth curve. At the cusp that $\mathbb{CP}^1 \times \mathbb{CP}^1$ creates on the elliptic curve, 5 to 18% of the nearby cells fail at every depth, and some of the others glue with the wrong orientation. The Kähler test removes the latter, but a correct extraction at a singular point needs a different method.
- Continuation can miss a small component of $C \cap c$ that meets no 2-face of c . A reducible curve needs one starting cell per component.

The most natural next step is certification. A per-cell test in the spirit of Plantinga and Vegter, adapted to codimension two, can use a single complex condition: that an enclosure of $\langle \nabla F(x), \nabla F(x_0) \rangle$ over the cell, with x_0 its center, excludes 0. With the implicit function theorem and the Noshiro–Warschawski theorem, this makes $C \cap c$ the graph of a holomorphic function over a complex line. On geodesic faces F is no longer a polynomial in the face coordinates, so the enclosures must come from interval arithmetic rather than Bernstein coefficients. Other open directions are curves of degree well beyond 6, the Kähler test as a general consistency check in $\mathbb{CP}^2$, and other complex surfaces that admit small balanced seeds.

Note on AI assistance

The implementation described here was developed with Claude (Anthropic) acting as a coding assistant, under the author’s direction, review and testing. This manuscript was likewise prepared with Claude’s assistance.

Code availability. The mesh data structures and the example programs are part of VGTL [34]. The LPT codes and their extension to balanced seeds, with the test suites of Section 4.2, are in the `lpt` repository [33]. Scripts regenerating all figures and tables are in VGTL’s `report/article/figures`.

Funding. This research received no specific funding.

References

- [1] M. Alkämper, F. Gaspoz, and R. Klöfkorn. A weak compatibility condition for newest vertex bisection in any dimension. *SIAM J. Sci. Comput.*, 40(6):A3853–A3872, 2018.
- [2] E. L. Allgower and K. Georg. *Introduction to Numerical Continuation Methods*, volume 45 of *Classics in Applied Mathematics*. SIAM, 2003.
- [3] E. L. Allgower and P. H. Schmidt. An algorithm for piecewise-linear approximation of an implicitly defined manifold. *SIAM J. Numer. Anal.*, 22(2):322–346, 1985.

- [4] F. B. Atalay and D. M. Mount. Pointerless implementation of hierarchical simplicial meshes and efficient neighbor finding in arbitrary dimensions. *Internat. J. Comput. Geom. Appl.*, 17(6):595–631, 2007.
- [5] G. Belda-Ferrín, E. Ruiz-Gironés, A. Gargallo-Peiró, and X. Roca. Conformal marked bisection for local refinement of n -dimensional unstructured simplicial meshes. *Computer-Aided Design*, 154:103419, 2023. verify volume.
- [6] P. Bhaniramka, R. Wenger, and R. Crawfis. Isosurface construction in any dimension using convex hulls. *IEEE Trans. Vis. Comput. Graph.*, 10(2):130–141, 2004.
- [7] A. I. Bobenko and C. Klein, editors. *Computational Approach to Riemann Surfaces*, volume 2013 of *Lecture Notes in Mathematics*. Springer, 2011.
- [8] J.-D. Boissonnat, S. Kachanovich, and M. Wintraecken. Tracing isomanifolds in $\mathbb{R}^d$ in time polynomial in d using Coxeter–Freudenthal–Kuhn triangulations. *SIAM J. Comput.*, 2023. SoCG 2021 version: LIPIcs 189:17.
- [9] J.-D. Boissonnat and M. Wintraecken. The topological correctness of PL approximations of isomanifolds. *Found. Comput. Math.*, 22:967–1012, 2022. verify pages.
- [10] D. A. Brake, D. J. Bates, W. Hao, J. D. Hauenstein, A. J. Sommese, and C. W. Wampler. Algorithm 976: Bertini_real: numerical decomposition of real algebraic curves and surfaces. *ACM Trans. Math. Softw.*, 44(1), 2017.
- [11] C. Burstedde and J. Holke. A tetrahedral space-filling curve for nonconforming adaptive meshes. *SIAM J. Sci. Comput.*, 38(5):C471–C503, 2016.
- [12] C. Burstedde, L. C. Wilcox, and O. Ghattas. p4est: scalable algorithms for parallel adaptive mesh refinement on forests of octrees. *SIAM J. Sci. Comput.*, 33(3):1103–1133, 2011.
- [13] A. Castelo, L. G. Nonato, M. Siqueira, R. Minghim, and G. Tavares. The J1a triangulation: an adaptive triangulation in any dimension. *Computers & Graphics*, 30(5):737–753, 2006.
- [14] B. Deconinck and M. van Hoeij. Computing Riemann matrices of algebraic curves. *Physica D*, 152–153:28–46, 2001.
- [15] L. Dening, L. Gehring, and J. Storn. Adaptive mesh refinement for arbitrary initial triangulations. *Found. Comput. Math.*, 2025.
- [16] J. Dupuy. Concurrent binary trees (with application to longest edge bisection). *Proc. ACM Comput. Graph. Interact. Tech.*, 3(2):21:1–21:20, 2020.
- [17] Seth Dutter. Visualization of complex projective curves. *arXiv:2608.04323*, 2026.
- [18] Asla Medeiros e Sá, Vinícius Moreira Mello, Karina Rodriguez Echavarria, and Derek Covill. Adaptive voids: primal and dual adaptive cellular structures for additive manufacturing. *The Visual Computer*, 31(6–8):799–808, 2015.
- [19] F. Effenberger and J. Spreer. simpcomp – a GAP toolbox for simplicial complexes. *ACM Commun. Comput. Algebra*, 44(4):186–189, 2010.
- [20] W. Evans, D. Kirkpatrick, and G. Townsend. Right-triangulated irregular networks. *Algorithmica*, 30(2):264–286, 2001.
- [21] A. A. Gaifullin. A minimal triangulation of complex projective plane admitting a chess colouring of four-dimensional simplices. *Proc. Steklov Inst. Math.*, 266:29–48, 2009.

- [22] T. Gerstner and M. Rumpf. Multiresolutional parallel isosurface extraction based on tetrahedral bisection. In *Volume Graphics*. Springer, 2000.
- [23] Phillip Griffiths and Joseph Harris. *Principles of Algebraic Geometry*. Wiley, New York, 1978.
- [24] A. J. Hanson. A construction for computer visualization of certain complex curves. *Notices Amer. Math. Soc.*, 41(9):1156–1163, 1994.
- [25] D. J. Hebert. Symbolic local refinement of tetrahedral grids. *J. Symbolic Comput.*, 17(5):457–472, 1994.
- [26] Y. Ju, X. Du, Q. Zhou, N. Carr, and T. Ju. Adaptive grid generation for discretizing implicit complexes. *ACM Trans. Graph.*, 43(4), 2024.
- [27] I. Kossaczky. A recursive approach to local mesh refinement in two and three dimensions. *J. Comput. Appl. Math.*, 55(3):275–288, 1994.
- [28] S. Kranich. *Continuity in Dynamic Geometry: An Algorithmic Approach*. PhD thesis, TU München, 2016.
- [29] W. Kühnel and T. F. Banchoff. The 9-vertex complex projective plane. *Math. Intelligencer*, 5(3):11–22, 1983.
- [30] M. Lee, L. De Floriani, and H. Samet. Constant-time neighbor finding in hierarchical tetrahedral meshes. In *Proc. Shape Modeling International*, pages 286–295, 2001.
- [31] J. M. Maubach. Local bisection refinement for n -simplicial grids generated by reflection. *SIAM J. Sci. Comput.*, 16(1):210–227, 1995.
- [32] J. M. Maubach. The efficient location of neighbors for locally refined n -simplicial grids. In *Proc. 5th International Meshing Roundtable*, pages 137–153, 1996.
- [33] Vinícius Mello. `lpt`: pointerless `lpt` codes for hierarchical simplicial meshes, with the `glpt` extension to balanced seeds. <https://github.com/vinicius-mello/lpt>, 2026.
- [34] Vinícius Mello. `VGTL`: a generic `c++` library for simplicial and semi-simplicial complexes. <https://github.com/vinicius-mello/vgtl2.0>, 2026.
- [35] W. F. Mitchell. *Unified Multilevel Adaptive Finite Element Methods for Elliptic Problems*. PhD thesis, University of Illinois, 1988.
- [36] W. F. Mitchell. Adaptive refinement for arbitrary finite-element spaces with hierarchical bases. *J. Comput. Appl. Math.*, 36(1):65–78, 1991.
- [37] B. Mourrain and J.-P. Pavone. Subdivision methods for solving polynomial equations. *J. Symbolic Comput.*, 44(3):292–306, 2009.
- [38] M. Nieser, K. Poelke, and K. Polthier. Automatic generation of Riemann surface meshes. In *Advances in Geometric Modeling and Processing (GMP 2010)*, volume 6130 of *LNCS*, pages 161–178, 2010. verify pages.
- [39] V. Pascucci. Slow growing subdivision (SGS) in any dimension: towards removing the curse of dimensionality. In *Comput. Graph. Forum (Proc. Eurographics 2002)*, volume 21, 2002.
- [40] S. Plantinga and G. Vegter. Isotopic approximation of implicit curves and surfaces. In *Proc. Symposium on Geometry Processing*, pages 245–254, 2004.

- [41] M. Reuter, T. S. Mikkelsen, E. C. Sherbrooke, T. Maekawa, and N. M. Patrikalakis. Solving nonlinear polynomial systems in the barycentric Bernstein basis. *The Visual Computer*, 24:187–200, 2008.
- [42] R. Stevenson. The completion of locally refined simplicial partitions created by bisection. *Math. Comp.*, 77(261):227–241, 2008.
- [43] C. T. Traxler. An algorithm for adaptive mesh refinement in n dimensions. *Computing*, 59(2):115–137, 1997.
- [44] C. Weigle and D. C. Banks. Complex-valued contour meshing. In *Proc. IEEE Visualization '96*, pages 173–180, 1996. verify pages.
- [45] K. Weiss and L. De Floriani. Simplex and diamond hierarchies: models and applications. *Comput. Graph. Forum*, 30(8):2127–2155, 2011. verify pages.